AI Engineering and Claude Architect · Part III, Agents

Agents and MCP, Lesson 6

Multi-Agent Orchestration: Anthropic's Research System

The report on "AI's impact on hospitals" was excellent on radiology, and silent on billing and nursing. Every subagent had done its job well. The coordinator had asked them the wrong question, and the bug was invisible unless you knew what was missing.

Module	AI06 of 13
Lesson	6 of 10
Prerequisites	AI06 L1–L3, L5, AI05 L4
Exam relevance	**Direct.** All of TS 1.2 and TS 1.3, and Scenario 3
Reading time	~50 minutes

Contents

1. Multi-Agent Orchestration

Lesson 6 of 10 · Agents and MCP

Multi-Agent Orchestration

Key idea

Multi-agent systems trade tokens for parallelism and separate context windows. Anthropic reports about 15× the tokens of a chat, so they fit only high-value work that splits cleanly. **Subagents inherit nothing. Everything they need goes in their prompt, and everything flows through the coordinator.** When output is poor, look at the decomposition before blaming the workers.

> *Before: orchestrator-workers (L1), the loop (L2), tool scope (L3), errors (L5), provenance (AI05 L4). Teaches: the published research system, coordinator design, context passing, parallel spawning, iterative refinement. Used by: AI06 L10, AI07, AI12 L3.*

The hook: every worker succeeded, and the report failed

A coordinator splits the topic, and each subagent succeeds. Then the coverage check runs. Real output from `code/ai06-agents/.../ResearchCoordinator.java`. Subagents are simulated with invented findings, and no model is called:

```
Round 1 ran [radiology, triage], gaps now [billing, nursing]
Round 2 ran [billing, nursing], gaps now []

radiology  OK       1 finding(s), tried: web and journal search
triage     OK       1 finding(s), tried: web and journal search
billing    PARTIAL  1 finding(s), tried: journal search worked; payer portal timed out twice
nursing    OK       1 finding(s), tried: web and journal search

Coverage notes for the report:
  billing is partial: journal search worked; payer portal timed out twice
```

Key idea

Narrow decomposition is the coordinator's bug, not the subagents'. So **TS 1.2** tests whether you can tell that a downstream agent worked correctly on a poor assignment. The fix belongs at the layer that made the assignment, and the coverage check gives the system a second chance.

Intuition: an investigative newsroom

An editor assigns stories to reporters: one on court records, one on interviews, one on financial filings. Reporters don't read each other's notebooks, and all findings go through the editor.

Newsroom	Agent system
Editor decides who covers what	Coordinator decomposes and delegates
A clear brief per reporter	Complete prompt per subagent
"Check what the others did" won't work, since they can't see it	Isolated contexts
Editor spots a missing angle, and sends someone back	Iterative refinement
Quotes keep their sources	Claim-source mappings

A vague brief ("look into the company") gets two reporters on the same court filing.

How Anthropic solved it

This is one of the few places Anthropic has published details of a real production system: How we built our multi-agent research system. Everything in this section reports that post.

The architecture

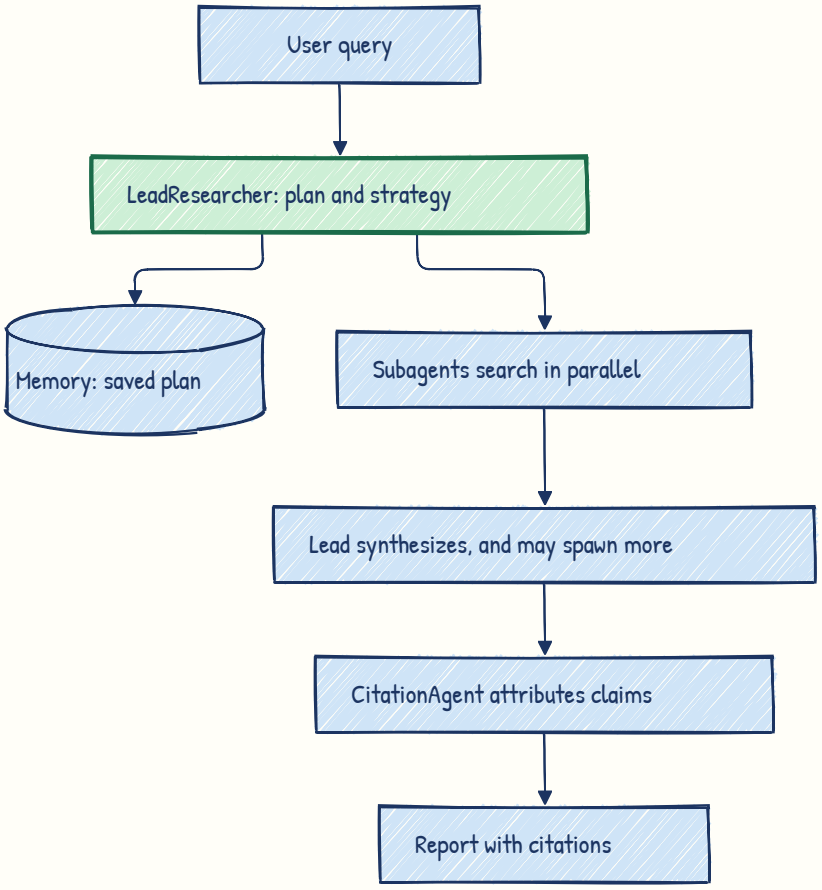


Figure 1. Anthropic's published research architecture. A lead agent plans and saves its plan to memory, spawns subagents that search and return findings, and a separate citation agent attributes claims at the end.

The lead saves its plan to memory because the context is truncated beyond 200,000 tokens, and the plan must survive that. Unlike static RAG, this searches iteratively and adapts.

What they measured

Finding	Figure	How to read it
Multi-agent vs single-agent, on an internal research eval	**90.2% better**	Opus 4 lead with Sonnet 4 subagents, against single-agent Opus 4
Tokens compared with chat	Agents **about 4×**, multi-agent **about 15×**	So it needs high-value tasks
What explains performance (BrowseComp)	3 factors explain **95%** of variance. Token usage alone explains **80%**	Spending more, across separate contexts, is much of the point
Parallelization	Lead runs **3–5** subagents, each uses **3+** tools. Research time cut by **up to 90%** on complex queries	Parallel at both levels
Tool-testing agent rewrote descriptions	**40%** less task time later	Tool quality compounds
Early eval set	About **20 queries**, enough to see a prompt tweak move success from 30% to 80%	Large effects show up in small samples

These are Anthropic's numbers, on its tasks and models of that time. Don't expect them elsewhere.

When it fits, per the post: heavy parallelization, information exceeding one context window, and many complex tools. **When it doesn't:** domains needing shared context or many dependencies between agents, such as most coding tasks.

Their eight prompting principles

#	Principle	Detail
1	Think like your agents	Simulations showed agents continuing after they had enough, and using wrong tools
2	**Teach the orchestrator to delegate**	Each subagent needs an **objective, output format, tool and source guidance, and task boundaries**
3	**Scale effort to complexity**	Simple fact: 1 agent, 3–10 calls. Comparison: 2–4 subagents, 10–15 calls each. Complex: more than 10 subagents
4	Tool design is critical	"Bad tool descriptions can send agents down completely wrong paths."
5	Let agents improve themselves	Claude diagnosed failures, and suggested fixes
6	Start wide, then narrow	Short, broad queries first
7	Guide the thinking process	Lead plans in thinking. Subagents assess results after each tool call
8	Parallel tool calling	See above

Principle 2 has a telling example from the post: with vague instructions, one subagent studied the 2021 chip crisis, while two others duplicated work on 2025 supply chains.

Overall, they favor "good heuristics rather than rigid rules", with guardrails against runaway behavior.

Evaluation and production

Area	What they did
Outcome over path	Valid paths vary, so evaluate outcomes and a reasonable process
LLM judge	Rubric: factual accuracy, citation accuracy, completeness, source quality, tool efficiency. One call giving 0.0–1.0 scores plus pass or fail aligned best with humans
Human testing	Found agents preferring SEO content farms over authoritative sources
Resuming	Resume from the point of failure, with retries and checkpoints
Tracing	Full production tracing, monitoring patterns without reading conversations, for privacy
Deployment	Rainbow deployments, since agents may be running during updates
A remaining bottleneck	The lead waits for each batch, and can't steer subagents
The telephone game	Subagents store output externally, and pass lightweight references to reduce loss

Mechanism: the coordinator pattern

The exam's model is **hub-and-spoke**.

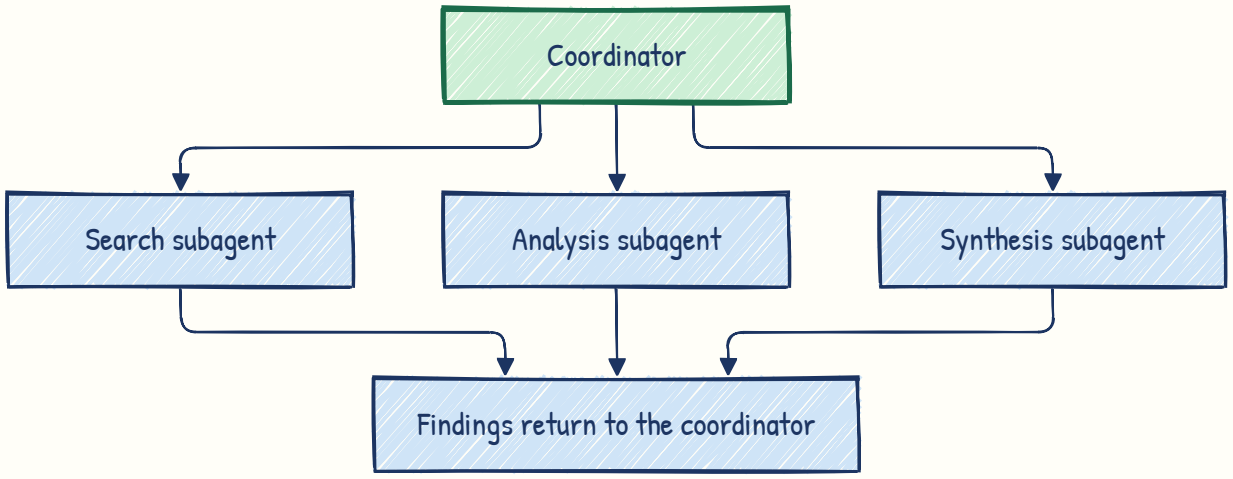


Figure 2. Hub and spoke. Subagents never talk to each other. The coordinator handles routing, errors and what information goes where, which makes the system observable.

The coordinator…	Rather than…
Analyzes the query, and **selects** which subagents to run	Always running the full pipeline
Partitions scope by subtopic or source type	Giving overlapping assignments
States goals and quality criteria	Dictating step-by-step procedures
Evaluates synthesis for gaps, and re-delegates	Accepting the first pass
Handles errors and routes all messages	Letting subagents call each other

Context passing: the part people get wrong

Subagents inherit nothing. This Agent SDK table is precise:

A subagent receives	A subagent doesn't receive
Its own system prompt, and the Agent tool's prompt string	The parent's conversation or tool results
Project `CLAUDE.md`, if settings load it	The parent's system prompt
Tool definitions: inherited, or the subset named	Skills, unless listed

"The only content you pass from parent to subagent is the Agent tool's prompt string." (A forked subagent is the exception: it copies the conversation.) So the prompt must be complete. Here's what one looks like, real output from the lab:

```
Goal: How is AI changing hospital operations?
Your part: triage only. Others cover the other aspects.
Already known, so don't repeat it: nothing yet
Return: JSON findings with claim, source URL and publication date.
Stop when: you have 3 independent sources, or after 10 searches.
If a source fails: say which, and return what you have.
```

Each line maps to Anthropic's principle 2: objective, boundaries, output format, and limits. And to **TS 1.3**: pass complete prior findings in the prompt, using structured data to keep content apart from metadata, such as URLs and page numbers (AI05 L4).

Spawning: the exam and today

Topic	Exam guide	Current docs
Spawning tool	**Task**	Renamed **Agent** in Claude Code 2.1.63. It still appears as `Task` in the `system:init` list, so match both
`allowedTools`	Must include `"Task"`	Include `"Agent"`
Parallel	Multiple Task calls in **one** response	Same idea
Waiting	Implied	By default, a subagent runs in the background, and its report arrives later. Claude waits when its next step needs it

```
# From the Agent SDK docs, using "Agent". The exam guide's version says "Task".
ClaudeAgentOptions(
    allowed_tools=["Read", "Grep", "Glob", "Agent"],
    agents={
        "code-reviewer": AgentDefinition(
            description="Expert code review specialist. Use for quality and security.",
            prompt="You are a code review specialist...",
            tools=["Read", "Grep", "Glob"],
        ),
    },
)
```

`AgentDefinition`'s core fields: `description` (when to use it), `prompt` (its system prompt), `tools` (its restriction). Limits you should set: `CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH` (default 3), `CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS` (default 20), and `max_budget_usd`. Note that this SDK is Python and TypeScript, so a Java service would use the Messages API for its own orchestration (AI08).

Parallel means one response. Spawning across separate turns makes them sequential.

Iterative refinement

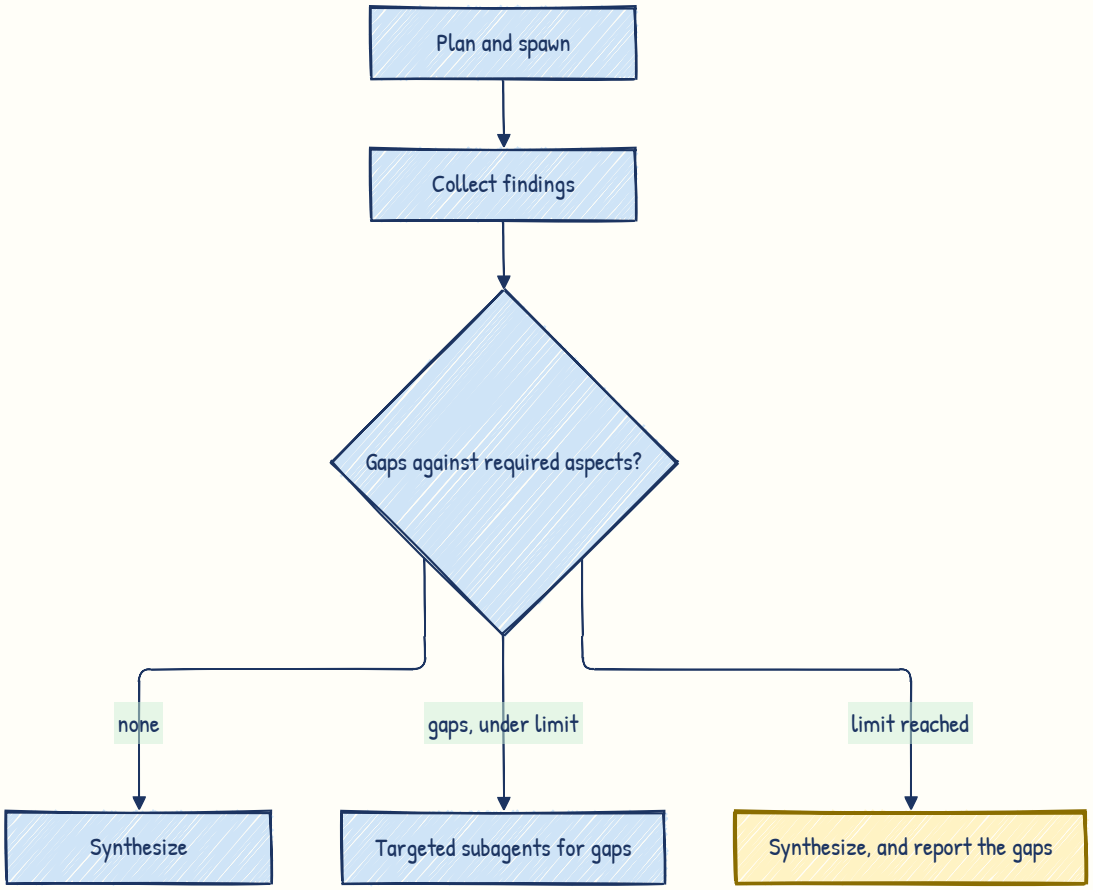


Figure 3. The refinement loop. The coordinator checks for gaps and sends targeted requests, stopping when coverage is enough, or at a limit, where it reports what's missing.

Why the obvious fixes fall short

Symptom	Tempting fix	Real fix
Report misses subtopics	Improve the search subagent	Fix the coordinator's decomposition
Duplicate work	Add subagent-to-subagent chat	Partition scope clearly
Synthesis lacks data	Tell it to "use all findings"	Pass the findings **in its prompt**
Sources lost	Ask synthesis to add citations	Structured mappings (AI05 L4)
Slow	Bigger model	Several spawns in one response
Quality low	More agents	Check the eval, since one agent might do

The production architecture

This is a recommended reference design, not a specific company's system. It uses ideas from Anthropic's post, and AI06 L10 expands it.

Baseline: one agent doing everything. It's simple, and it fills its context with search results.

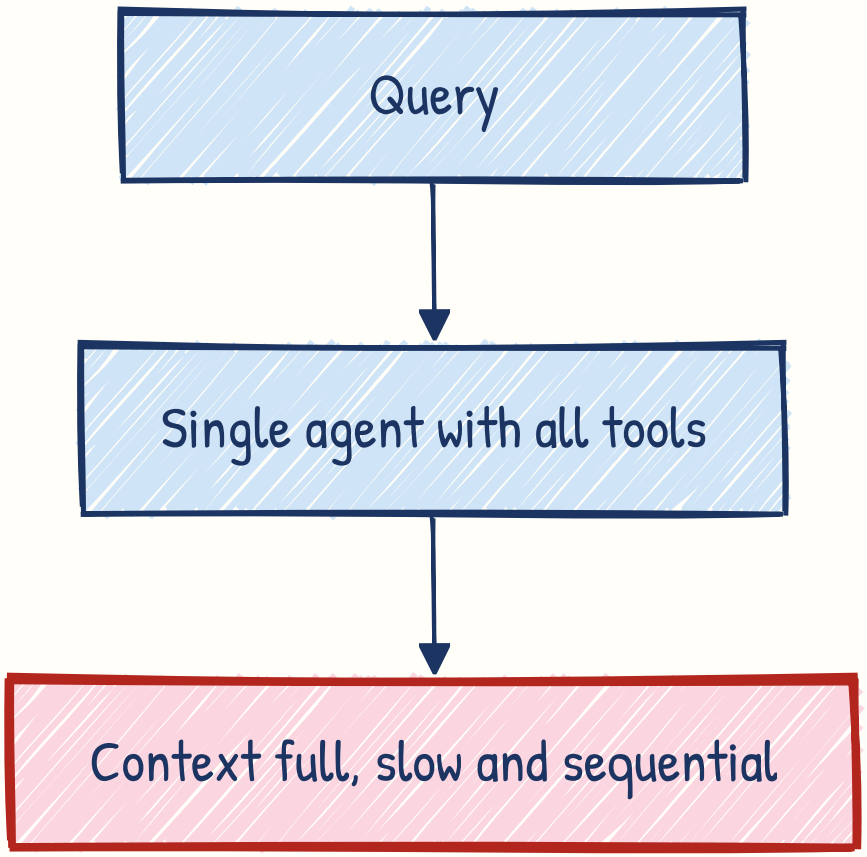


Figure 4. Baseline. One agent reads everything itself, so context fills up and searches run one after another.

Improved: a fixed pipeline. Search, analyze, synthesize, every time. New problem: **simple questions pay for the whole pipeline, and hard ones can't adapt.**

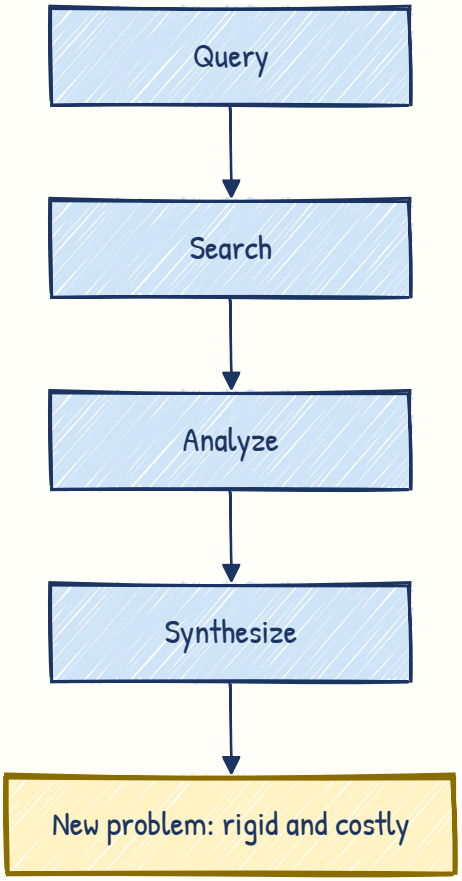


Figure 5. Improved. A fixed pipeline separates the work, but it's rigid and costly on easy questions.

Production: a dynamic coordinator with bounds.

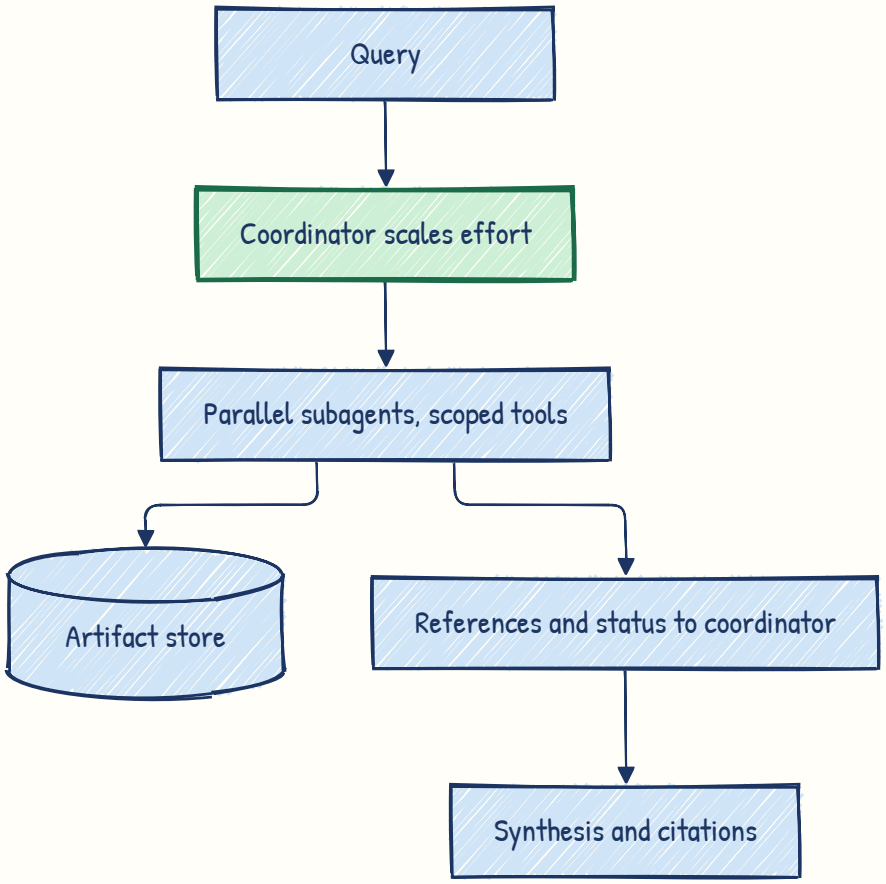


Figure 6. Production. The coordinator scales effort per query and spawns in parallel. Subagents write artifacts and return references, and a budget bounds the run.

Component	Why it exists	How it communicates
Coordinator	Plans, selects, checks gaps	Spawns through a tool, with complete prompts
Subagents	Separate contexts, in parallel	Isolated. Return final message only
Artifact store	Avoids the telephone game	References passed back
Budget	Multi-agent costs can explode	Dollars and depth limits

Failures

Failure	Containment
One subagent fails	Partial results and coverage notes (AI06 L5)
Spawn explosion	Depth, concurrency and budget limits
Subagents duplicate each other	Clear boundaries, and "already known" in the prompt
Coordinator's context fills	Save the plan externally, and return references
Runaway searching	Stop criteria in the prompt, and a limit

Java

Parallel subagents with virtual threads, from `ResearchCoordinator.java`. In a Java service you'd make these calls through the Messages API.

```
try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<SubResult>> running = new ArrayList<>();
    for (String aspect : aspects) {
        running.add(pool.submit(() -> subagent(goal, aspect, known)));
    }
```

Real output: `Subagents: 4. Sequential would sleep 1200 ms. This run took about 800 ms (rounded to 100).` The ideal is 600 ms across two rounds of 300 ms, and the rest is overhead. Each sleep stands in for a subagent, so this shows concurrency only, and says nothing about real speed-ups.

Trade-offs

Decision	Gain	Cost
Multi-agent	Parallelism, separate contexts	About 15× chat's tokens, per Anthropic
Hub and spoke	Observable, consistent error handling	Coordinator is a bottleneck
Isolated contexts	Focus	You must pass everything explicitly
Goals rather than procedures	Adaptability	Less predictable
Artifact references	Fidelity, fewer tokens	A store to manage

When not to use it: tasks with heavy dependencies between parts, shared context, or low value per task. Anthropic names most coding tasks here.

Common mistakes

1. **Assuming subagents share memory.** They don't.
2. **Spawning across turns for parallelism.** Use one response.
3. **Forgetting `Task` or `Agent` in `allowedTools`.** The coordinator can't spawn.
4. **Blaming workers for narrow scope.** Check the coordinator.
5. **Always running the full pipeline.** Select per query.
6. **Step-by-step coordinator prompts.** State goals and criteria.
7. **Multi-agent for coding by default.** Shared context matters there.
8. **Quoting 90.2% as a general result.** It's one internal eval.

What to say in an interview

"Anthropic's research system uses a lead that plans and spawns parallel subagents, each with its own context, and a citation pass at the end. It reports 90.2% better on an internal eval, at about 15× chat's tokens, so I'd use the pattern only for high-value, parallel work. Subagents inherit nothing, so each prompt carries the objective, boundaries, format and any prior findings. If a report misses topics, I'd first check the coordinator's decomposition."

Certification lens

How this lesson maps to CCAR-F

TS 1.2, **TS 1.3**, and Scenario 3. Hub-and-spoke, isolated context, explicit prompts, parallel calls in one response, goals rather than procedures, and root-cause decomposition. The guide says Task. Current products say Agent.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Multi-Agent Research System · Medium. A report on "renewable energy" covers only solar and wind. Logs show the coordinator assigned just those two subtopics, and both subagents performed well. What's the root cause?

A. The search subagent's queries were too narrow.

B. The synthesis subagent dropped other findings.

C. The web search tool's index lacks other sources.

D. The coordinator's decomposition was too narrow.

A **D** (TS 1.2). The logs show subagents did what they were asked. A and B blame downstream agents, and C has no evidence.

Q Q2 · Exam style, scenario: Multi-Agent Research System · Medium. The synthesis subagent says that it has no findings to work with, though search has already returned results. What's the fix?

A. Include the complete search findings in the synthesis subagent's prompt.

B. Have synthesis read the coordinator's conversation history.

C. Enable shared memory between subagents.

D. Have search send findings directly to synthesis.

A **A** (TS 1.3). Subagents inherit nothing. B and C assume features that don't work that way. D breaks hub-and-spoke routing.

Q Q3 · Exam style, scenario: Multi-Agent Research System · Medium. To run three search subagents in parallel, what should the coordinator do?

A. Spawn one per turn, waiting for each.

B. Emit three subagent tool calls in a single response.

C. Start three separate coordinators.

D. Ask one subagent to do three searches.

A **B** (TS 1.3). A is sequential. C loses coordination. D loses separate contexts.

Q Q4 · Exam style, scenario: Multi-Agent Research System · Hard. Choose the **two** changes that most reduce duplicated work and missed coverage.

A. Assign distinct subtopics or source types to each subagent.

B. Evaluate synthesis output for gaps, and re-delegate targeted queries.

C. Let subagents message each other to coordinate.

D. Run every subagent twice, and merge.

E. Always invoke all subagents, for every query.

A **A and B** (TS 1.2). C breaks routing through the coordinator. D wastes tokens and duplicates by design. E ignores query complexity.

Q Q5 · Exam style, scenario: Developer Productivity with the Agent SDK · Easy. A coordinator's subagents never run, though `AgentDefinition`s exist. `allowedTools` lists `Read` and `Grep`. Per the guide, what's missing?

A. A `--parallel` flag.

B. A larger `max_tokens`.

C. The `Task` tool in `allowedTools`.

D. A `subagents.json` file.

A **C** (TS 1.3). Today it's named `Agent`. A and D don't exist. B is unrelated.

Revision - Lesson 6

→ Multi-agent trades tokens for parallelism and separate contexts.

→ Anthropic: agents about 4 times, multi-agent about 15 times chat's tokens.

→ Anthropic: 90.2% better on one internal eval. Up to 90% less time with parallel calls.

→ Fits: parallel, large information, many tools. Poor fit: shared context, such as most coding.

→ Lead saves its plan to memory. A citation agent attributes claims.

→ Delegate with objective, format, tool guidance and boundaries.

→ Scale effort: 1 agent for a fact, and more than 10 for complex research.

→ Hub and spoke: everything goes through the coordinator.

→ Subagents inherit nothing, so pass complete findings in the prompt.

→ Parallel means several spawn calls in one response.

→ Exam says Task, and current products say Agent.

→ Coordinators select subagents, state goals, and refine against gaps.

→ Poor coverage usually means poor decomposition.

Your notes